



Práctica

Presentación

Por fin llegamos al final del curso. Esta práctica servirá para sintetizar todos los conocimientos del curso y ampliarlos con el diseño de circuitos más complejos. En esta práctica se os pedirá el diseño de un circuito a partir de un grafo de estados y el diseño de una máquina de estados en concreto. Para la primera parte, tendréis que aplicar los conocimientos que habéis adquirido en este curso como, por ejemplo, los mapas de Karnaugh o los sistemas de representación. Por lo tanto, se recomienda que les deis un vistazo.

Competencias

Conocer la organización general de un computador como circuito digital y conocer los rasgos distintivos de la arquitectura de Von Neumann.

Objetivos

- Conocer varios modelos de las máquinas de estados y de las arquitecturas de controlador con camino de datos.
- Haber adquirido una experiencia básica en la elección del modelo de máquina de estados más adecuado para la resolución de un problema concreto.
- Ser capaz de diseñar circuitos secuenciales a partir de grafos de transiciones de estados.

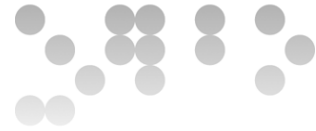
Recursos

Los recursos que se recomienda usar para esta práctica son los siguientes:

- **Básicos:** El módulo 5 de los materiales.
- **Complementarios:** VerilCIRC, VerilCHART y el Wiki de la asignatura.

Criterios de valoración

- Razonad la respuesta en todos los ejercicios. Las respuestas no justificadas no recibirán puntuación.
- La valoración está indicada en cada uno de los subapartados.



Formato y fecha de entrega

- Para dudas y aclaraciones sobre el enunciado, dirigíos al consultor responsable de vuestra aula.
- Hay que entregar la solución en un fichero PDF usando una de las plantillas entregadas junto con este enunciado.
- Se tiene que entregar a través de la aplicación de **Entrega y registro de EC** del apartado Evaluación de vuestra aula.
- La fecha límite de entrega es el **27 de mayo**, a las 24 horas.

Solución de la práctica propuesta

PRIMERA PARTE [65%]

Dada la máquina de estados siguiente:

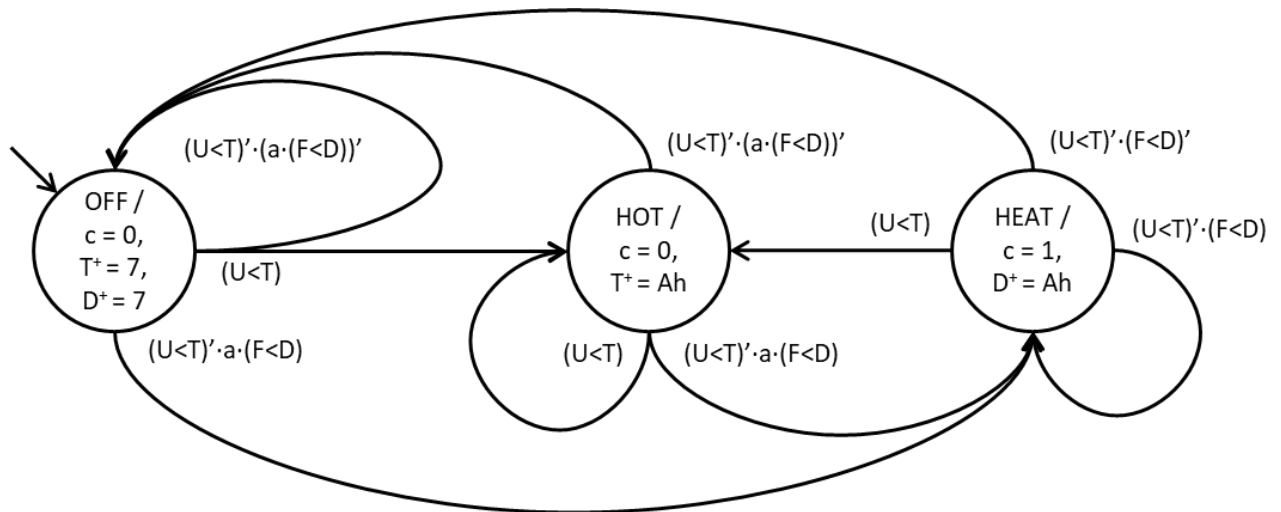
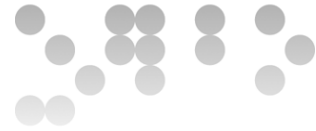


Fig. 1. EFSM de un controlador de una caldera.

Se pide que:

- a) [15%] A partir del grafo de la EFSM de la fig. 1 completad el cronograma siguiente, con la evolución de D , T , c y del estado (E) en que se encuentra la máquina en cada ciclo de reloj.

Nota: Tenéis el ejercicio disponible en VerilChart, donde el estado es E y los valores de D y de T , que tienen 4 bits, se representan en hexadecimal.



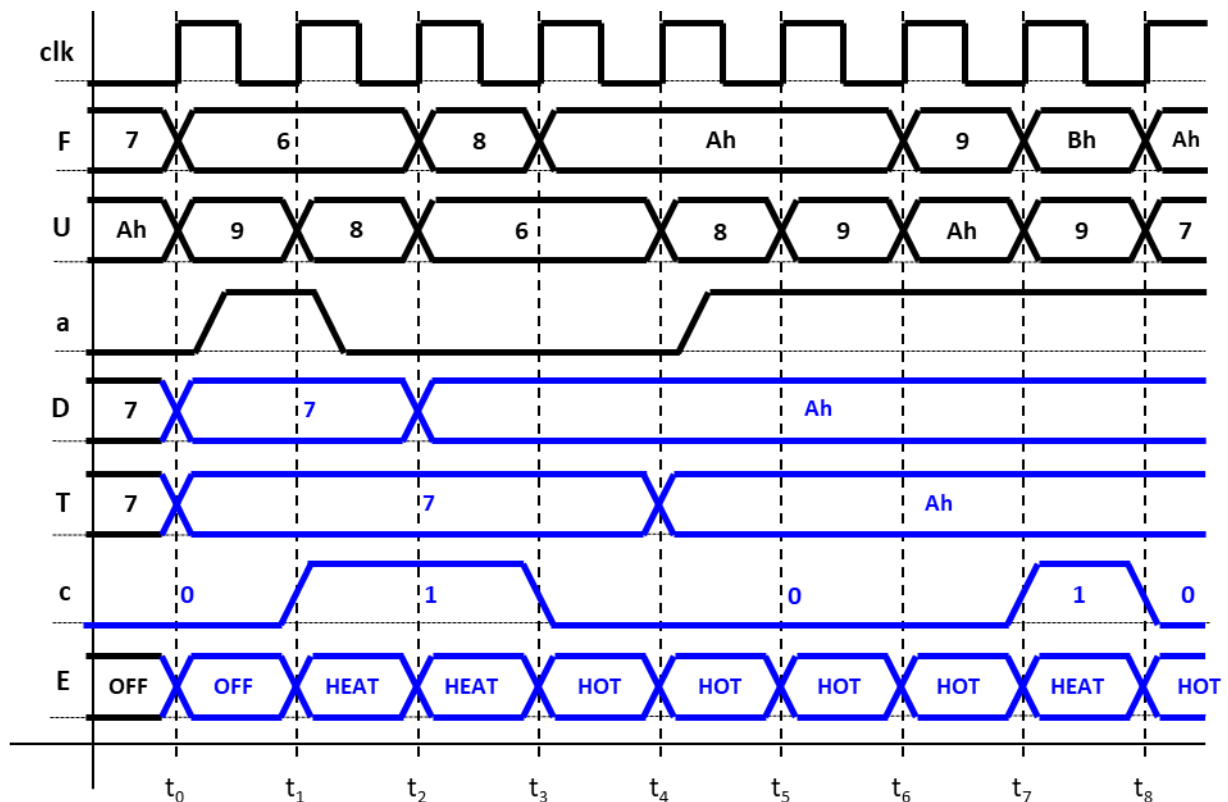
A partir de las condiciones iniciales que hay en el cronograma ($E=OFF$, $D=7$, y $T=7$) se puede determinar el estado siguiente y los nuevos valores para D y T para el periodo t_0-t_1 .

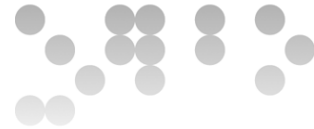
El valor de la señal c depende de forma exclusiva y combinacional del estado E . Así pues, sabiendo que inicialmente la máquina está en OFF, el valor de $c = 0$.

En el caso de los valores para D y T , serán ambos 7 porque, en OFF, se indica que D y T , en el estado siguiente, valdrán 7.

Determinar el valor del estado siguiente implica determinar cuál de las transiciones se activa. En este caso, $(U < T)$ es falso porque A_h no es menor que 7 y, por lo tanto, $(U < T)'$ es cierto. Esto quiere decir que sólo se podrían activar las transiciones hacia HEAT o hacia OFF. Como $a=0$, la única que se cumple es $(U < T)' \cdot (a \cdot (F < D))'$ y, por lo tanto, el valor del estado siguiente será OFF.

Siguiendo este procedimiento, se pueden determinar los valores para el periodo t_1-t_2 a partir de los del periodo anterior y se puede ir repitiendo el procedimiento hasta completar el cronograma, tal como se muestra en la figura siguiente.





- b) [15%] Obtened las tablas de transiciones y de salidas de la EFSM de la fig. 1, así como las codificaciones binarias de estados y salidas correspondientes.

En el grafo hay tres señales de entrada: a , $(F < D)$ y $(U < T)$. Además, están las variables D y T y la señal de salida c , para las que hay que determinar qué valores toman.

Las tablas de transiciones y salidas que se obtienen a partir del grafo son:

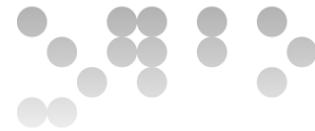
Estado actual	Entrada	Estado siguiente	Estado	Salidas
OFF	$(U < T)' \cdot (a \cdot (F < D))'$	OFF	OFF	$c = 0, T^+ = 7, D^+ = 7$
OFF	$(U < T)$	HOT	HOT	$c = 0, T^+ = Ah, D^+ = D$
OFF	$(U < T)' \cdot a \cdot (F < D)$	HEAT	HEAT	$c = 1, T^+ = T, D^+ = Ah$
HOT	$(U < T)' \cdot (a \cdot (F < D))'$	OFF		
HOT	$(U < T)$	HOT		
HOT	$(U < T)' \cdot a \cdot (F < D)$	HEAT		
HEAT	$(U < T)' \cdot (F < D)'$	OFF		
HEAT	$(U < T)$	HOT		
HEAT	$(U < T)' \cdot (F < D)$	HEAT		

Para la codificación binaria, hay que asignar en cada estado un código binario individual. En este caso, se sigue el criterio más habitual, que es el de la numeración binaria, desde 0 para el estado inicial hasta el número de estados que haya menos 1: 00 para OFF, 01 para HOT y 10 para HEAT.

Para elegir el valor siguiente que se asigna a las variables D y T hacen falta dos selectores de dos bits cada uno para poder elegir entre tres valores posibles: 7, Ah y el valor actual. Como cada valor se elige en un estado diferente, se puede usar el código de estado como selector:

Operaciones	Estado	Efecto sobre la variable
$T^+ = 7, D^+ = 7$	00	Ponerlas a 7
$T^+ = Ah, D^+ = D$	01	Asignar Ah a T y mantener el valor de D
$T^+ = T, D^+ = Ah$	10	Asignar Ah a D y mantener el valor de T

El valor de la salida c se determina combinatorialmente a partir del estado. El valor que tiene se puede obtener directamente del código del estado: coincide con el bit de más peso.



Finalmente, las tablas de transiciones y de salidas con la codificación binaria correspondiente son:

Estado actual		Entrada			Estado ⁺
Símbolo	q_1q_0	(U<T)	a	(F<D)	$q_1^+q_0^+$
OFF	00	0	x	0	00
OFF	00	0	0	x	00
OFF	00	0	1	1	10
OFF	00	1	x	x	01
HOT	01	0	x	0	00
HOT	01	0	0	x	00
HOT	01	0	1	1	10
HOT	01	1	x	x	01
HEAT	10	0	x	0	00
HEAT	10	0	x	1	10
HEAT	10	1	x	x	01

Estado		Selector
Símbolo	q_1q_0	q_1q_0
OFF	00	00
HOT	01	01
HEAT	10	10

Notad que, en la columna “Selector” se ha repetido el contenido del estado. Para seleccionar los nuevos valores de D y T se usa el código del estado y para determinar el valor de la salida c se usa q_1 , el bit de más peso del código del estado. Si se usan señales diferentes a estas, también es correcto. (Tened presente, además, que, según la codificación elegida para los estados y para el selector, el resultado de la tabla anterior puede variar.)

El cálculo de la raíz cuadrada entera de un número X se puede representar con la EFSM siguiente.

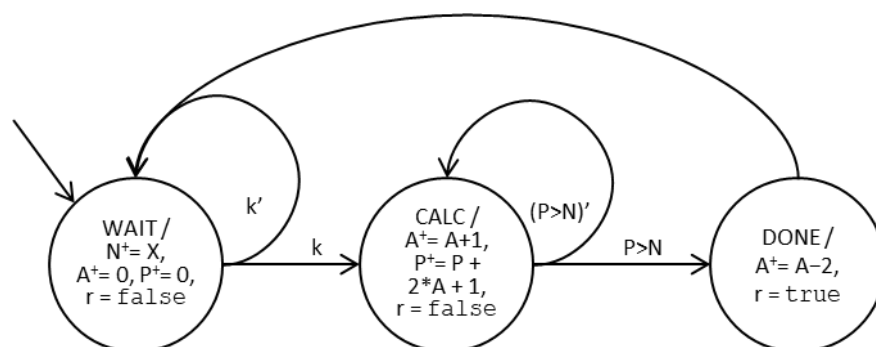
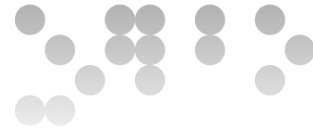


Fig. 2. EFSM del cálculo de la raíz cuadrada entera de X .



En cuanto al funcionamiento, cada vez que la entrada k (petición de cálculo) se pone a 1, se pasa a calcular la raíz cuadrada entera de X que se almacena en N . Después, se pone la salida r (resultado disponible) a 1 para indicar que la raíz cuadrada se puede leer de la variable A en el estado siguiente.

Para facilitar las tareas de verificación, la entrada X es de sólo 4 bits.

Teniendo en cuenta lo que se ha indicado previamente, se pide que:

- c) [15%] A partir de las tablas de transiciones y de salidas del grafo de la fig. 2 que se dan a continuación, implementad la unidad de control usando ROM y los registros, los bloques combinacionales y las puertas lógicas que creáis convenientes. Especificad y justificad las dimensiones de la/las ROM que uséis e indicad el contenido en binario, indicando a que corresponde cada bit, y en hexadecimal.

Estado actual	Entrada		Estado*
	k	$(P>N)$	
WAIT	0	x	WAIT
WAIT	1	x	CALCO
CALCO	x	0	CALCO
CALCO	x	1	DOY
DOY	x	x	WAIT

Estado		Salidas			
Símbolo	q_1q_0	r	ld_N	s_A	s_P
WAIT	01	0	1	01	01
CALCO	00	0	0	00	00
DOY	10	1	0	10	10

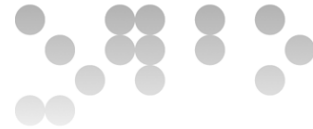
Notas: La codificación que se ha dado a los estados permite simplificar el cálculo de las salidas. Tenéis el ejercicio disponible en VerilCIRC y, si queréis comprobar previamente correcto funcionamiento de la ROM, la tenéis disponible en VerilChart. En ese entorno, el estado es E y $(P>N)$ es PN .

Para implementar la unidad de control con una ROM hace falta, primero, determinar el tamaño y, después, calcular el contenido a partir de la tabla de transiciones correspondiente.

El bus de direcciones de la ROM tiene que ser de 4 bits, puesto que la dirección se forma con los bits que codifican el estado (son 2, en este caso) y los de las señales de entrada (2 más). Por lo tanto, la ROM tiene que disponer de un total de $2^4 = 16$ posiciones de memoria.

En cada posición se almacena el código del estado siguiente y el valor, en el estado actual, de cada una de las señales de salida. Por lo tanto, hacen falta 2 bits para el estado y harían falta 6 bits para las salidas (r , ld_N , s_A y s_P), lo que haría un total de 8 bits.

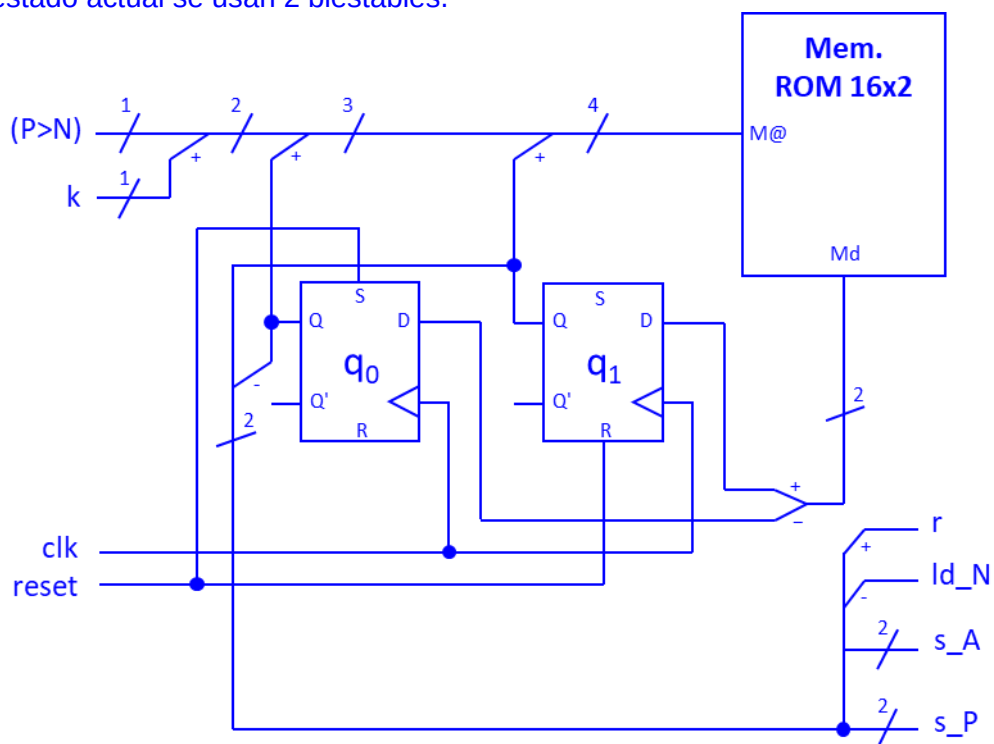
De todos modos, se puede observar que los selectores s_A y s_P coinciden con el valor del estado actual y, por lo tanto, no hay que almacenarlos en la ROM. Lo mismo pasa con la salida r , que es el bit más significativo del código del estado actual, q_1 , y con ld_N , que coincide con el bit menos significativo, q_0 .

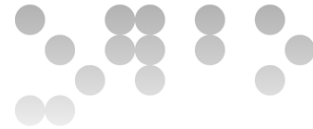


La ROM, pues, debe ser sólo para el cálculo del estado siguiente, de una dimensión de 16 palabras de 2 bits. El contenido de la ROM se determina partir de la tabla de transiciones:

Posición de memoria			Contenido	Codificación hexadecimal
q_1q_0	k	(P>N)	$q_1^+q_0^+$	
00	0	0	00	0h
00	0	1	10	2h
00	1	0	00	0h
00	1	1	10	2h
01	0	0	01	1h
01	0	1	01	1h
01	1	0	00	0h
01	1	1	00	0h
10	0	0	01	1h
10	0	1	01	1h
10	1	0	01	1h
10	1	1	01	1h
11	0	0	00	0h
11	0	1	00	0h
11	1	0	00	0h
11	1	1	00	0h

La figura siguiente muestra la implementación con la ROM correspondiente. La entrada $M@$ de la ROM se configura con el valor del estado actual (los 2 bits de más peso) y el valor de las entradas (los 2 bits de menos peso). Para guardar el estado actual se usan 2 biestables.





Hay que tener en cuenta que las señales de selección y de salida se obtienen directamente del estado actual, es decir, de las salidas de los biestables q_0 y q_1 y no de la memoria ROM.

Además, la señal de *reset* está conectada a la entrada S (*set*) del biestable q_0 y en la entrada R (*reset*) de q_1 porque el estado inicial WAIT tiene código 01.

- d) [20%] Implementad el circuito completo de la EFSM de la fig. 2: Construid el camino de datos usando las puertas lógicas y los bloques necesarios e incorporad la unidad de control obtenida en el apartado anterior. Indicad y razonad, para cada bus, su dimensión en bits teniendo en cuenta que la señal de entrada X es de 4 bits y que representa un valor sin signo (natural).

Nota: Tenéis el ejercicio disponible en VerilCIRC. En el circuito, la señal P también es de salida para poder comprobar más fácilmente el funcionamiento. Tened en cuenta que VerilCIRC no soporta la verificación de subcircuitos diseñados por el usuario. Por lo tanto, tendréis que copiar en vuestra solución el circuito de la unidad de control que hayáis diseñado en el apartado anterior.

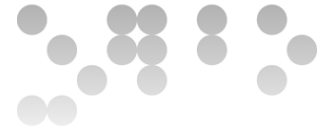
Por un lado, el camino de datos debe ocuparse de generar las señales necesarias para la unidad de control. En este caso, sólo hay que generar la señal ($P > N$). De otro lado, el camino de datos también tiene que actualizar y almacenar el valor de las variables que se usen y generar los resultados de las operaciones que sean necesarias. Aquí se usan las variables N , A , y P .

La entrada X es un valor natural (es decir, sin signo) de 4 bits. Por lo tanto, N , que es la variable que guarda el valor de X , tiene que ser de 4 bits.

Del grafo podemos deducir que la variable A , que finalmente tendrá el resultado de la raíz cuadrada entera de X , puede llegar a tener el valor de la raíz cuadrada de X más 2. El valor máximo de X es 15 y esto comporta que A puede llegar a tener el valor 5. Consiguientemente, con 3 bits es suficiente para la variable A .

La máquina pasa del estado CALC al estado DONE cuando P supera el valor de N . El valor máximo de N es 15. En P se van sucediendo los valores del cuadrado de los números naturales (0, 1, 4, 9, ...). El primer valor que supera 15 es el valor 16. En ese momento, se pasa en el estado DONE. Por lo tanto, se necesitan 5 bits para la variable P . De hecho, aún falta una nueva actualización de P (porque es una variable). Pero antes de usar de nuevo el valor de P se habrá inicializado a 0 y, por lo tanto, no sería preocupante que esta última actualización de P , que pasaría a tener el valor 25, pudiera provocar un desbordamiento. (En este caso, como X es de 4 bits no produce ningún desbordamiento. Esta situación solo se daría si X fuera una entrada de 1, 2 o 3 bits. Para 4 o más bits no se produce desbordamiento.)

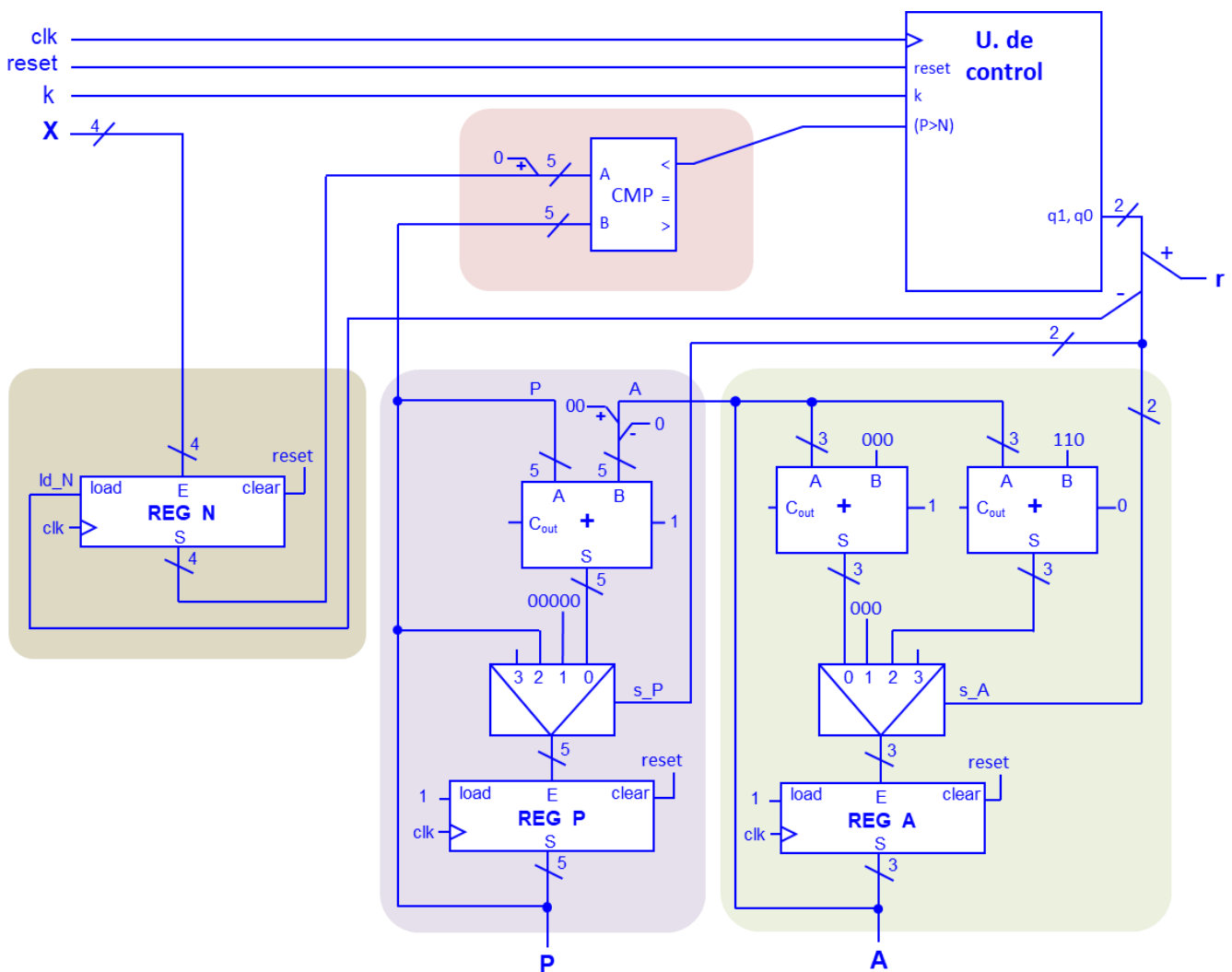
Según el estado, la variable A puede tener que tomar el valor 0, incrementarse en 1, valor que podemos obtener con un sumador, o decrementarse en 2, operación por la que también usaremos un sumador (sumaremos el valor -2 , en $Ca2$ y 3 bits, el 110). Un multiplexor controlado por el código del estado servirá para determinar cuál de estos valores llega a la entrada del registro A .

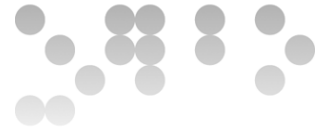


Al registro que guarda el valor de P tienen que poder llegar el valor resultante de la operación $P+2*A+1$, que podemos obtener con sólo un sumador (porque $2*A$ se consigue añadiendo un bit 0 como bit de menos peso de A), el valor 0, o el mismo valor que tenga P . De nuevo, un multiplexor controlado por el código del estado nos sirve para determinar cuál es el nuevo valor de P .

Por último, la carga del registro N se controla con el bit de menos peso del código del estado, q_0 . Con la codificación de estados que se ha hecho, eso quiere decir que la carga del registro sólo estará habilitada cuando la máquina esté en el estado WAIT, código 01.

Con esto, el circuito completo para la EFSM de la fig. 2 queda de la forma siguiente:





SEGUNDA PARTE [35%]

Hay que diseñar el modelo de un controlador de un semáforo de paso (ATLC) para regular la entrada de personas de forma que ayude a mantener la distancia física. La persona que esté en primer lugar en la cola de entrada podrá pasar cuando esté en verde. El semáforo solo se pondrá en verde cuando detecte que hay una persona en la cola (cámara B) y no detecte a nadie en la rampa de entrada (cámara A). Una vez que se pone en verde, se mantendrá hasta que la cámara A detecte alguna persona. En aquel momento, volverá a posarse en rojo como mínimo durante 10 segundos y hasta que no se cumplan las condiciones para volver a ponerse en verde.

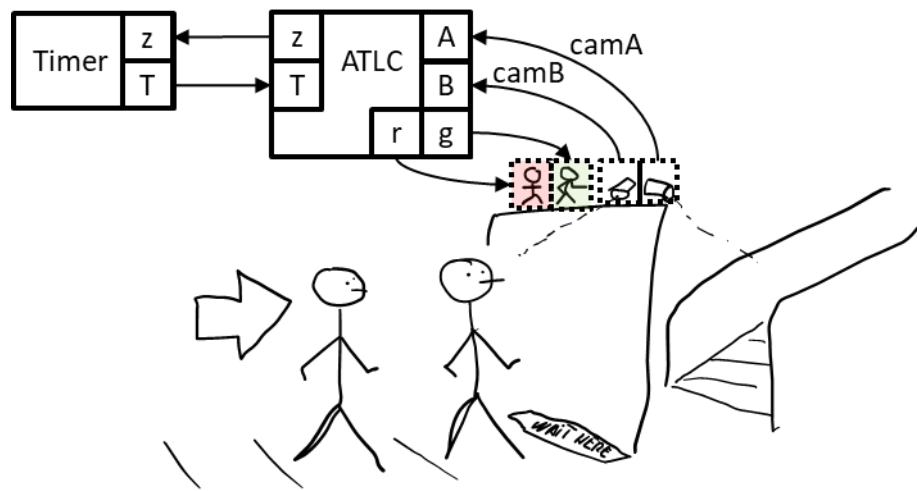


Fig. 3. Esquema de bloques del controlador del semáforo espaciador de personas.

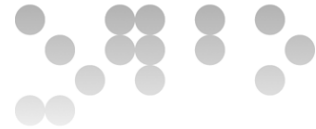
El controlador tiene las entradas y salidas siguientes:

- Las entradas *A* y *B*, que proporcionan un valor entre 0 y 100 en cuanto a la probabilidad de que haya una persona en las imágenes. La presencia de personas se considerará cierta cuando el valor de la probabilidad sea superior o igual a 5;
- La entrada *T*, que **proporciona el tiempo en número de segundos** que ha transcurrido desde la última vez que se ha puesto a cero;
- La salida *z*, que hace que el *Timer* (temporizador) ponga *T* a 0 un ciclo después de que *z* deje de tener el valor 1, y
- Las salidas *r* y *g*, que se tienen que poner a 1 para encender las luces rojo y verde, respectivamente (y a 0 para mantenerlas apagadas).

Diseñad la ASM del controlador del semáforo de espaciamiento entre personas que se ha descrito.

El controlador debe ocuparse de generar las señales *r* y *g*, para mantener encendidas o apagadas las luces del semáforo, y la señal *z* para inicializar el temporizador.

El control se basa en la información que se recibe de las cámaras. Por lo tanto, hace falta un estado inicial en el que el semáforo se pone en rojo y, a partir de aquí, evaluar la información que llega de las cámaras para determinar si el semáforo tiene que ponerse en verde o mantenerse en rojo.



Cuando, eventualmente, el semáforo se pone en verde y posteriormente en rojo, porque se detecta alguna persona en la rampa, la señal *z* inicializa el *Timer*, para poder garantizar que el semáforo se mantenga en rojo un mínimo de 10 segundos.

La figura siguiente muestra la ASM correspondiente a ese comportamiento:

